
Experiment 4

Implementation of Newton–Raphson, Fast Decoupled, in MATLAB

Lab Report

Student Name HAMZA AHMAD

Registration Number 2022-EE-165

Section B

Power System Operation and Control Laboratory

Fall 2025

MATLAB code with comments

https://github.com/hamza-2274/PSOC/blob/main/newton_raphson.m

```
%% =====  
% NEWTON-RAPHSON LOAD FLOW ANALYSIS (IEEE-14 Bus System)  
%  
% This program performs power flow analysis using Newton-Raphson method  
% for the IEEE 14-bus test system.  
% =====
```

```

clear; clc; close all;

fprintf('\n--- NEWTON-RAPHSON LOAD FLOW (IEEE-14 BUS) ---\n\n');

%% ===== SYSTEM DATA =====

BMva = 100;    % Base MVA

% Bus Data Format:

% Columns: Bus | Vsp | delta | Pg | Qg | Pl | Ql | Qmin | Qmax | Type
% Bus Type: 1=Slack, 2=PV (Generator), 3=PQ (Load)

busdata = [
    1   1.060   0   232.4   -16.9    0.0    0.0    0    0    1;
    2   1.045   0    40.0   43.56   21.7   12.7   -40   50    2;
    3   1.010   0     0.0    0.0   94.2   19.0   -25   40    3;
    4   1.000   0     0.0    0.0   47.8   -3.9    0    0    3;
    5   1.000   0     0.0    0.0    7.6    1.6    0    0    3;
    6   1.070   0     0.0    0.0   11.2    7.5    -6   24    2;
    7   1.000   0     0.0    0.0    0.0    0.0    0    0    3;
    8   1.090   0     0.0    0.0    0.0    0.0    0    0    2;
    9   1.000   0     0.0    0.0   29.5   16.6    0    0    3;
   10   1.000   0     0.0    0.0    9.0    5.8    0    0    3;
   11   1.000   0     0.0    0.0    3.5    1.8    0    0    3;
   12   1.000   0     0.0    0.0    6.1    1.6    0    0    3;
   13   1.000   0     0.0    0.0   13.5    5.8    0    0    3;
   14   1.000   0     0.0    0.0   14.9    5.0    0    0    3;

];

% Line Data Format:

% Columns: From | To | R | X | B/2 | Tap

linedata = [
    1   2   0.01938   0.05917   0.0264   1.000;
    1   5   0.05403   0.22304   0.0246   1.000;
    2   3   0.04699   0.19797   0.0219   1.000;
    2   4   0.05811   0.17632   0.0170   1.000;
    2   5   0.05695   0.17388   0.0173   1.000;
    3   4   0.06701   0.17103   0.0064   1.000;
    4   5   0.01335   0.04211   0.0000   1.000;

```

```

4   7   0.00000   0.20912   0.0000   0.978;   % Transformer
4   9   0.00000   0.55618   0.0000   0.969;   % Transformer
5   6   0.00000   0.25202   0.0000   1.000;
6  11   0.09498   0.19890   0.0000   1.000;
6  12   0.12291   0.25581   0.0000   1.000;
6  13   0.06615   0.13027   0.0000   1.000;
7   8   0.00000   0.17615   0.0000   1.000;
7   9   0.00000   0.11001   0.0000   1.000;
9  10   0.03181   0.08450   0.0000   1.000;
9  14   0.12711   0.27038   0.0000   1.000;
10  11   0.08205   0.19207   0.0000   1.000;
12  13   0.22092   0.19988   0.0000   1.000;
13  14   0.17093   0.34802   0.0000   1.000;

];

%% ===== Y-BUS FORMATION =====

fprintf('Building Y-bus matrix...\n');

nbus = max(max(linedata(:, 1:2)));

Y = zeros(nbus, nbus);

fb = linedata(:, 1);   % From bus
tb = linedata(:, 2);   % To bus

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);

    R = linedata(k, 3);
    X = linedata(k, 4);
    Bc = linedata(k, 5);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end

    Z = R + 1i * X;

```

```

y = 1 / Z;
ysh = 1i * Bc;

% Off-diagonal elements
Y(i, j) = Y(i, j) - y / tap;
Y(j, i) = Y(i, j);

% Diagonal elements
Y(i, i) = Y(i, i) + y / (tap^2) + ysh;
Y(j, j) = Y(j, j) + y + ysh;
end

G = real(Y); % Conductance matrix
B = imag(Y); % Susceptance matrix

%% ===== INITIALIZATION =====

type = busdata(:, 10);
V = busdata(:, 2);
del = deg2rad(busdata(:, 3));
Pg = busdata(:, 4) / BMva;
Qg = busdata(:, 5) / BMva;
Pl = busdata(:, 6) / BMva;
Ql = busdata(:, 7) / BMva;
Qmin = busdata(:, 8) / BMva;
Qmax = busdata(:, 9) / BMva;
Psp = Pg - Pl; % net active power injection
Qsp = Qg - Ql; % net reactive power injection
pq = find(type == 3); % PQ bus indices
npq = length(pq); % used to size the Jacobian submatrice
tol = 1e-6; % nvergence tolerance
Iter = 0;
Tol = 1;

fprintf('Starting Newton-Raphson iterations...\n\n');

%% ===== NEWTON-RAPHSON ITERATION =====

% Start timing for Newton-Raphson

```

```

tic_nr = tic;

while Tol > tol

    Iter = Iter + 1;

    % Calculate power injections

    P = zeros(nbus, 1);
    Q = zeros(nbus, 1);

    for i = 1:nbus
        for k = 1:nbus
            P(i) = P(i) + V(i) * V(k) * (G(i,k) * cos(del(i) - del(k)) + ...
                                           B(i,k) * sin(del(i) - del(k)));
            Q(i) = Q(i) + V(i) * V(k) * (G(i,k) * sin(del(i) - del(k)) - ...
                                           B(i,k) * cos(del(i) - del(k)));
        end
    end

    % Calculate mismatches

    dP = Psp - P;
    dQ = Qsp - Q;
    M = [dP(2:end); dQ(pq)];
    Tol = max(abs(M));

    if Tol < tol
        break;
    end

    % ===== JACOBIAN MATRIX CONSTRUCTION =====

    % J1: dP/d(delta)

    J1 = zeros(nbus - 1);

    for i = 1:nbus - 1
        m = i + 1;

```

```

for k = 1:nbus - 1
    n = k + 1;
    if n == m
        for n2 = 1:nbus
            J1(i, k) = J1(i, k) + V(m) * V(n2) * ...
                (-G(m,n2) * sin(del(m) - del(n2)) + ...
                B(m,n2) * cos(del(m) - del(n2)));
        end
        J1(i, k) = J1(i, k) - V(m)^2 * B(m, m);
    else
        J1(i, k) = V(m) * V(n) * ...
            (G(m,n) * sin(del(m) - del(n)) - ...
            B(m,n) * cos(del(m) - del(n)));
    end
end
end

% J2: dP/dV
J2 = zeros(nbus - 1, npq);
for i = 1:nbus - 1
    m = i + 1;
    for k = 1:npq
        n = pq(k);
        if n == m
            for n2 = 1:nbus
                J2(i, k) = J2(i, k) + V(n2) * ...
                    (G(m,n2) * cos(del(m) - del(n2)) + ...
                    B(m,n2) * sin(del(m) - del(n2)));
            end
            J2(i, k) = J2(i, k) + V(m) * G(m, m);
        else
            J2(i, k) = V(m) * ...
                (G(m,n) * cos(del(m) - del(n)) + ...

```

```

        B(m,n) * sin(del(m) - del(n)));

    end

end

end

% J3: dQ/d(delta)
J3 = zeros(npq, nbus - 1);

for i = 1:npq
    m = pq(i);

    for k = 1:nbus - 1
        n = k + 1;

        if n == m

            for n2 = 1:nbus
                J3(i, k) = J3(i, k) + V(m) * V(n2) * ...
                    (G(m,n2) * cos(del(m) - del(n2)) + ...
                    B(m,n2) * sin(del(m) - del(n2)));

            end

            J3(i, k) = J3(i, k) - V(m)^2 * G(m, m);

        else

            J3(i, k) = V(m) * V(n) * ...
                (-G(m,n) * cos(del(m) - del(n)) - ...
                B(m,n) * sin(del(m) - del(n)));

        end

    end

end

end

```

```

% J4: dQ/dV
J4 = zeros(npq);

for i = 1:npq
    m = pq(i);

    for k = 1:npq
        n = pq(k);

        if n == m

```

```

        for n2 = 1:nbus
            J4(i, k) = J4(i, k) + V(n2) * ...
                (G(m,n2) * sin(del(m) - del(n2)) - ...
                B(m,n2) * cos(del(m) - del(n2)));

        end

        J4(i, k) = J4(i, k) - V(m) * B(m, m);

    else

        J4(i, k) = V(m) * ...
            (G(m,n) * sin(del(m) - del(n)) - ...
            B(m,n) * cos(del(m) - del(n)));

    end

end

end

end

% Assemble full Jacobian
J = [J1 J2; J3 J4];

% Solve for corrections
X = J \ M;

% Update state variables
dTheta = X(1:nbus - 1);
dV = X(nbus:end);
del(2:end) = del(2:end) + dTheta;
V(pq) = V(pq) + dV;

end

% Stop timing for Newton-Raphson
time_nr = toc(tic_nr);

%% ===== NR RESULTS =====

fprintf('\n');

fprintf('=====\n');

fprintf(' NEWTON-RAPHSON: CONVERGENCE ACHIEVED\n');

fprintf('=====\n');

```

```

fprintf(' Iterations: %d\n', Iter);
fprintf(' Max Mismatch: %.6e pu\n', Tol);
fprintf(' Computation Time: %.6f seconds\n', time_nr);
fprintf(' Time per Iteration: %.6f seconds\n', time_nr/Iter);
fprintf('=====\n\n');
Angles_deg = rad2deg(del);
fprintf('BUS VOLTAGE AND ANGLE RESULTS:\n');
fprintf('-----\n');
fprintf(' Bus | Voltage (pu) | Angle (deg) \n');
fprintf('-----\n');
for i = 1:nbus
    fprintf(' %2d | %7.4f | %9.4f\n', i, V(i), Angles_deg(i));
end
fprintf('-----\n');
%% ===== NR POWER LOSS CALCULATION =====
fprintf('\nCALCULATING TRANSMISSION LOSSES...\n\n');
Ploss_total = 0;
Qloss_total = 0;
for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);
    R = linedata(k, 3);
    X = linedata(k, 4);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end

    Z = R + 1i * X;
    y = 1 / Z;

    Vi = V(i) * exp(1i * del(i));

```

```

Vj = V(j) * exp(1i * del(j));

Iij = (Vi - Vj / tap) * y;
Iji = (Vj - Vi * tap) * y;

Sij = Vi * conj(Iij);
Sji = Vj * conj(Iji);

Ploss_total = Ploss_total + real(Sij + Sji);
Qloss_total = Qloss_total + imag(Sij + Sji);
end

fprintf('=====\n');
fprintf('  SYSTEM LOSSES (NR)\n');
fprintf('=====\n');
fprintf('  Active Power Loss:   %.6f pu   (%.3f MW)\n', ...
        Ploss_total, Ploss_total * BMva);
fprintf('  Reactive Power Loss: %.6f pu   (%.3f MVar)\n', ...
        Qloss_total, Qloss_total * BMva);
fprintf('=====\n\n');
fprintf('=== NEWTON-RAPHSON SIMULATION COMPLETE ===\n\n');

```

RESULTS

Convergence plots (iterations vs mismatch)?

Tables of bus voltages and line flows?

```
=====
NEWTON-RAPHSON: CONVERGENCE ACHIEVED
=====
```

```
Iterations: 4
Max Mismatch: 1.021696e-07 pu
Computation Time: 0.230019 seconds
Time per Iteration: 0.057505 seconds
=====
```

BUS VOLTAGE AND ANGLE RESULTS:

```
-----
Bus | Voltage (pu) | Angle (deg)
-----
1 | 1.0600 | 0.0000
2 | 1.0450 | -4.9657
3 | 0.9856 | -12.4247
4 | 1.0162 | -10.3081
5 | 1.0265 | -8.9055
6 | 1.0700 | -14.7611
7 | 1.0514 | -13.3756
8 | 1.0900 | -13.3756
9 | 1.0350 | -14.9951
10 | 1.0336 | -15.2363
11 | 1.0401 | -15.1160
```

fx

Command window

8		1.0900		-13.3756
9		1.0350		-14.9951
10		1.0336		-15.2363
11		1.0481		-15.1169
12		1.0536		-15.5986
13		1.0473		-15.6265
14		1.0221		-16.2941

CALCULATING TRANSMISSION LOSSES...

=====

SYSTEM LOSSES (NR)

=====

Active Power Loss: 0.158841 pu (15.884 MW)|

Reactive Power Loss: 0.568311 pu (56.831 MVar)

=====

=== NEWTON-RAPHSON SIMULATION COMPLETE ===

CODE OF NR VS FAST DECOUPLED

```
%% =====
% NEWTON-RAPHSON LOAD FLOW ANALYSIS (IEEE-14 Bus System)
% WITH Y-BUS AND JACOBIAN MATRIX DISPLAY
% =====

clear; clc; close all;

fprintf('\n--- NEWTON-RAPHSON LOAD FLOW (IEEE-14 BUS) ---\n\n');

%% ===== SYSTEM DATA =====

BMva = 100; % Base MVA

% Bus Data Format:

% Columns: Bus | Vsp | delta | Pg | Qg | Pl | Ql | Qmin | Qmax | Type
% Bus Type: 1=Slack, 2=PV (Generator), 3=PQ (Load)

busdata = [
    1  1.060  0  232.4  -16.9  0.0  0.0  0  0  1;
    2  1.045  0  40.0   43.56 21.7  12.7 -40 50  2;
```

```

3   1.010   0       0.0    0.0   94.2   19.0   -25   40    3;
4   1.000   0       0.0    0.0   47.8   -3.9    0    0    3;
5   1.000   0       0.0    0.0    7.6    1.6    0    0    3;
6   1.070   0       0.0    0.0   11.2    7.5    -6   24    2;
7   1.000   0       0.0    0.0    0.0    0.0    0    0    3;
8   1.090   0       0.0    0.0    0.0    0.0    0    0    2;
9   1.000   0       0.0    0.0   29.5   16.6    0    0    3;
10  1.000   0       0.0    0.0    9.0    5.8    0    0    3;
11  1.000   0       0.0    0.0    3.5    1.8    0    0    3;
12  1.000   0       0.0    0.0    6.1    1.6    0    0    3;
13  1.000   0       0.0    0.0   13.5    5.8    0    0    3;
14  1.000   0       0.0    0.0   14.9    5.0    0    0    3;

```

```
];
```

```
% Line Data Format:
```

```
% Columns: From | To | R | X | B/2 | Tap
```

```
linedata = [
```

```

1   2   0.01938   0.05917   0.0264   1.000;
1   5   0.05403   0.22304   0.0246   1.000;
2   3   0.04699   0.19797   0.0219   1.000;
2   4   0.05811   0.17632   0.0170   1.000;
2   5   0.05695   0.17388   0.0173   1.000;
3   4   0.06701   0.17103   0.0064   1.000;
4   5   0.01335   0.04211   0.0000   1.000;
4   7   0.00000   0.20912   0.0000   0.978;  % Transformer
4   9   0.00000   0.55618   0.0000   0.969;  % Transformer
5   6   0.00000   0.25202   0.0000   1.000;
6  11   0.09498   0.19890   0.0000   1.000;
6  12   0.12291   0.25581   0.0000   1.000;
6  13   0.06615   0.13027   0.0000   1.000;
7   8   0.00000   0.17615   0.0000   1.000;
7   9   0.00000   0.11001   0.0000   1.000;
9  10   0.03181   0.08450   0.0000   1.000;
9  14   0.12711   0.27038   0.0000   1.000;

```

```

10 11    0.08205    0.19207    0.0000    1.000;
12 13    0.22092    0.19988    0.0000    1.000;
13 14    0.17093    0.34802    0.0000    1.000;

l;

%% ===== Y-BUS FORMATION =====

fprintf('Building Y-bus matrix...\n');

nbus = max(max(linedata(:, 1:2)));

Y = zeros(nbus, nbus);

fb = linedata(:, 1); % From bus
tb = linedata(:, 2); % To bus

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);

    R = linedata(k, 3);
    X = linedata(k, 4);
    Bc = linedata(k, 5);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end

    Z = R + 1i * X;
    y = 1 / Z;
    ysh = 1i * Bc;

    % Off-diagonal elements
    Y(i, j) = Y(i, j) - y / tap;
    Y(j, i) = Y(i, j);

    % Diagonal elements
    Y(i, i) = Y(i, i) + y / (tap^2) + ysh;
    Y(j, j) = Y(j, j) + y + ysh;

```

```

end

G = real(Y); % Conductance matrix
B = imag(Y); % Susceptance matrix

%% ===== DISPLAY Y-BUS MATRIX =====

fprintf('\n===== \n');
fprintf(' Y-BUS MATRIX (ADMITTANCE MATRIX) \n');
fprintf('===== \n \n');
fprintf('Y-BUS in Rectangular Form (Real + j*Imag): \n \n');
for i = 1:nbus
    fprintf('Bus %2d: ', i);
    for j = 1:nbus
        if abs(Y(i,j)) > 1e-6
            fprintf(['%7.4f%+7.4fj ', real(Y(i,j)), imag(Y(i,j))]);
        else
            fprintf([' 0.0000 +0.0000j ']);
        end
    end
    fprintf('\n');
end

fprintf('\n \n CONDUCTANCE MATRIX (G = Real part of Y): \n \n');
for i = 1:nbus
    fprintf('Bus %2d: ', i);
    for j = 1:nbus
        fprintf('%8.4f ', G(i,j));
    end
    fprintf('\n');
end

fprintf('\n \n SUSCEPTANCE MATRIX (B = Imaginary part of Y): \n \n');
for i = 1:nbus
    fprintf('Bus %2d: ', i);
    for j = 1:nbus
        fprintf('%8.4f ', B(i,j));
    end
end

```

```

        fprintf('\n');
end

fprintf('\n===== \n\n');

%% ===== INITIALIZATION =====

type = busdata(:, 10);
V = busdata(:, 2);
del = deg2rad(busdata(:, 3));
Pg = busdata(:, 4) / BMva;
Qg = busdata(:, 5) / BMva;
Pl = busdata(:, 6) / BMva;
Ql = busdata(:, 7) / BMva;
Qmin = busdata(:, 8) / BMva;
Qmax = busdata(:, 9) / BMva;
Psp = Pg - Pl; % net active power injection
Qsp = Qg - Ql; % net reactive power injection
pq = find(type == 3); % PQ bus indices
npq = length(pq);
tol = 1e-6; % convergence tolerance
Iter = 0;
Tol = 1;

fprintf('Starting Newton-Raphson iterations...\n\n');

%% ===== NEWTON-RAPHSON ITERATION =====

% Start timing for Newton-Raphson
tic_nr = tic;

% Flag to display Jacobian only once (first iteration)
display_jacobian = true;

while Tol > tol

    Iter = Iter + 1;

    % Calculate power injections

    P = zeros(nbus, 1);

    Q = zeros(nbus, 1);

```

```

for i = 1:nbus
    for k = 1:nbus
        P(i) = P(i) + V(i) * V(k) * (G(i,k) * cos(del(i) - del(k)) + ...
            B(i,k) * sin(del(i) - del(k)));
        Q(i) = Q(i) + V(i) * V(k) * (G(i,k) * sin(del(i) - del(k)) - ...
            B(i,k) * cos(del(i) - del(k)));
    end
end

```

```

% Calculate mismatches

```

```

dP = Psp - P;
dQ = Qsp - Q;
M = [dP(2:end); dQ(pq)];
Tol = max(abs(M));

```

```

if Tol < tol
    break;
end

```

```

% ===== JACOBIAN MATRIX CONSTRUCTION =====

```

```

% J1: dP/d(delta)

```

```

J1 = zeros(nbus - 1);
for i = 1:nbus - 1
    m = i + 1;
    for k = 1:nbus - 1
        n = k + 1;
        if n == m
            for n2 = 1:nbus
                J1(i, k) = J1(i, k) + V(m) * V(n2) * ...
                    (-G(m,n2) * sin(del(m) - del(n2)) + ...
                    B(m,n2) * cos(del(m) - del(n2)));
            end
        end
    end
end

```

```

        J1(i, k) = J1(i, k) - V(m)^2 * B(m, m);
    else
        J1(i, k) = V(m) * V(n) * ...
            (G(m,n) * sin(del(m) - del(n)) - ...
            B(m,n) * cos(del(m) - del(n)));
    end
end
end

end

% J2: dP/dV
J2 = zeros(nbus - 1, npq);
for i = 1:nbus - 1
    m = i + 1;
    for k = 1:npq
        n = pq(k);
        if n == m
            for n2 = 1:nbus
                J2(i, k) = J2(i, k) + V(n2) * ...
                    (G(m,n2) * cos(del(m) - del(n2)) + ...
                    B(m,n2) * sin(del(m) - del(n2)));
            end
            J2(i, k) = J2(i, k) + V(m) * G(m, m);
        else
            J2(i, k) = V(m) * ...
                (G(m,n) * cos(del(m) - del(n)) + ...
                B(m,n) * sin(del(m) - del(n)));
        end
    end
end
end

% J3: dQ/d(delta)
J3 = zeros(npq, nbus - 1);
for i = 1:npq

```

```

m = pq(i);
for k = 1:nbus - 1
    n = k + 1;
    if n == m
        for n2 = 1:nbus
            J3(i, k) = J3(i, k) + V(m) * V(n2) * ...
                (G(m,n2) * cos(del(m) - del(n2)) + ...
                B(m,n2) * sin(del(m) - del(n2)));
        end
        J3(i, k) = J3(i, k) - V(m)^2 * G(m, m);
    else
        J3(i, k) = V(m) * V(n) * ...
            (-G(m,n) * cos(del(m) - del(n)) - ...
            B(m,n) * sin(del(m) - del(n)));
    end
end
end

% J4: dQ/dV
J4 = zeros(npq);
for i = 1:npq
    m = pq(i);
    for k = 1:npq
        n = pq(k);
        if n == m
            for n2 = 1:nbus
                J4(i, k) = J4(i, k) + V(n2) * ...
                    (G(m,n2) * sin(del(m) - del(n2)) - ...
                    B(m,n2) * cos(del(m) - del(n2)));
            end
            J4(i, k) = J4(i, k) - V(m) * B(m, m);
        else
            J4(i, k) = V(m) * ...

```

```

        (G(m,n) * sin(del(m) - del(n)) - ...
        B(m,n) * cos(del(m) - del(n)));

    end

end

end

% ===== DISPLAY JACOBIAN MATRICES (FIRST ITERATION) =====
if display_jacobian
    fprintf('\n===== \n');
    fprintf('  JACOBIAN MATRICES (Iteration 1) \n');
    fprintf('===== \n \n');

    fprintf('J1 Matrix (dP/d_delta) [%dx%d]: \n', size(J1,1),
size(J1,2));

    fprintf('(Partial derivatives of real power w.r.t. voltage
angles) \n \n');

    disp(J1);

    fprintf('\nJ2 Matrix (dP/dV) [%dx%d]: \n', size(J2,1), size(J2,2));
    fprintf('(Partial derivatives of real power w.r.t. voltage
magnitudes) \n \n');

    disp(J2);

    fprintf('\nJ3 Matrix (dQ/d_delta) [%dx%d]: \n', size(J3,1),
size(J3,2));

    fprintf('(Partial derivatives of reactive power w.r.t. voltage
angles) \n \n');

    disp(J3);

    fprintf('\nJ4 Matrix (dQ/dV) [%dx%d]: \n', size(J4,1), size(J4,2));
    fprintf('(Partial derivatives of reactive power w.r.t. voltage
magnitudes) \n \n');

    disp(J4);

    fprintf('===== \n \n');

```

```

        display_jacobian = false;
    end

    % Assemble full Jacobian
    J = [J1 J2; J3 J4];

    % Solve for corrections
    X = J \ M;

    % Update state variables
    dTheta = X(1:nbus - 1);
    dV = X(nbus:end);
    del(2:end) = del(2:end) + dTheta;
    V(pq) = V(pq) + dV;
end

% Stop timing for Newton-Raphson
time_nr = toc(tic_nr);

%% ===== NR RESULTS =====
fprintf('\n');
fprintf('=====\n');
fprintf('  NEWTON-RAPHSON: CONVERGENCE ACHIEVED\n');
fprintf('=====\n');
fprintf('  Iterations: %d\n', Iter);
fprintf('  Max Mismatch: %.6e pu\n', Tol);
fprintf('  Computation Time: %.6f seconds\n', time_nr);
fprintf('  Time per Iteration: %.6f seconds\n', time_nr/Iter);
fprintf('=====\n\n');
Angles_deg = rad2deg(del);
fprintf('BUS VOLTAGE AND ANGLE RESULTS:\n');
fprintf('-----\n');
fprintf(' Bus | Voltage (pu) | Angle (deg) \n');
fprintf('-----\n');
for i = 1:nbus

```

```

        fprintf(' %2d | %7.4f | %9.4f\n', i, V(i), Angles_deg(i));
end

fprintf('-----\n');

%% ===== NR POWER LOSS CALCULATION =====

fprintf('\nCALCULATING TRANSMISSION LOSSES...\n\n');

Ploss_total = 0;
Qloss_total = 0;

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);
    R = linedata(k, 3);
    X = linedata(k, 4);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end

    Z = R + 1i * X;
    y = 1 / Z;

    Vi = V(i) * exp(1i * del(i));
    Vj = V(j) * exp(1i * del(j));

    Iij = (Vi - Vj / tap) * y;
    Iji = (Vj - Vi * tap) * y;

    Sij = Vi * conj(Iij);
    Sji = Vj * conj(Iji);

    Ploss_total = Ploss_total + real(Sij + Sji);
    Qloss_total = Qloss_total + imag(Sij + Sji);
end

```

```

fprintf('=====\n');
fprintf('  SYSTEM LOSSES (NR)\n');
fprintf('=====\n');
fprintf('  Active Power Loss:   %.6f pu   (%.3f MW)\n', ...
        Ploss_total, Ploss_total * BMva);
fprintf('  Reactive Power Loss: %.6f pu   (%.3f MVar)\n', ...
        Qloss_total, Qloss_total * BMva);
fprintf('=====\n\n');
fprintf('=== NEWTON-RAPHSON SIMULATION COMPLETE ===\n\n');

%% =====
%  FAST DECOUPLED LOAD FLOW IMPLEMENTATION
%  =====

fprintf('\n');

fprintf('=====\n');
fprintf('  STARTING FAST DECOUPLED LOAD FLOW ANALYSIS\n');
fprintf('=====\n\n');

%% =====  FDLF INITIALIZATION  =====

% Reset voltages to initial values
V_fd = busdata(:, 2);
del_fd = deg2rad(busdata(:, 3));

% B' matrix neglects resistance (only reactance considered)
B_prime = zeros(nbus, nbus);

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);

    R = linedata(k, 3);
    X = linedata(k, 4);
    Bc = linedata(k, 5);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end
end

```

```

% B' formation: neglect resistance, use only reactance
y_prime = 1 / (1i * X);

% Off-diagonal elements
B_prime(i, j) = B_prime(i, j) - imag(y_prime / tap);
B_prime(j, i) = B_prime(i, j);

% Diagonal elements
B_prime(i, i) = B_prime(i, i) + imag(y_prime / (tap^2));
B_prime(j, j) = B_prime(j, j) + imag(y_prime);
end

B_double_prime = zeros(nbus, nbus);
for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);
    X = linedata(k, 4);
    Bc = linedata(k, 5);
    tap = linedata(k, 6);
    if tap == 0
        tap = 1;
    end

    y = 1 / (1i * X);

    % Off-diagonal elements
    B_double_prime(i, j) = B_double_prime(i, j) - imag(y / tap);
    B_double_prime(j, i) = B_double_prime(i, j);

    % Diagonal elements (include shunt charging)
    B_double_prime(i, i) = B_double_prime(i, i) + imag(y / (tap^2)) + Bc/2;
    B_double_prime(j, j) = B_double_prime(j, j) + imag(y) + Bc/2;
end

```

```

%% ===== DISPLAY B' AND B'' MATRICES
=====

fprintf('\n===== \n');
fprintf(' B' (B-PRIME) MATRIX - For Real Power/Angle\n');
fprintf('===== \n');
fprintf('(Neglects resistance and shunt charging)\n\n');
for i = 1:nbus
    fprintf('Bus %2d: ', i);
    for j = 1:nbus
        fprintf('%8.4f ', B_prime(i,j));
    end
    fprintf('\n');
end
fprintf('\n\n===== \n');
fprintf(' B'' (B-DOUBLE-PRIME) MATRIX - For Reactive Power/Voltage\n');
fprintf('===== \n');
fprintf('(Neglects resistance, includes shunt charging)\n\n');
for i = 1:nbus
    fprintf('Bus %2d: ', i);
    for j = 1:nbus
        fprintf('%8.4f ', B_double_prime(i,j));
    end
    fprintf('\n');
end
fprintf('\n===== \n\n');
% Form Jacobian matrices
B11 = B_prime(2:end, 2:end);
B22 = B_double_prime(pq, pq);
fprintf('B11 (from B') [%dx%d] - Excluding slack bus:\n', size(B11,1), size(B11,2));
disp(B11);
fprintf('\nB22 (from B'') [%dx%d] - PQ buses only:\n', size(B22,1), size(B22,2));
disp(B22);

```

```

% Final Jacobian for Fast Decoupled
Jacob_FD = -[B11, zeros(nbus-1, npq);
             zeros(npq, nbus-1), B22];

tol_fd = 0.01;
Iter_fd = 0;
max_iter_fd = 100;

fprintf('Fast Decoupled method initialized...\n');
fprintf('Tolerance: %.6f\n\n', tol_fd);

%% ===== FDLF ITERATION =====

mismatch_fd = ones(2*nbus - 2 - length(find(type==2)), 1);
tic_fd = tic;

while any(abs(mismatch_fd(:)) > tol_fd) && Iter_fd < max_iter_fd
    Iter_fd = Iter_fd + 1;

    % Calculate power injections

    P_fd = zeros(nbus, 1);
    Q_fd = zeros(nbus, 1);

    for i = 1:nbus
        for k = 1:nbus
            P_fd(i) = P_fd(i) + V_fd(i) * V_fd(k) * ...
                (G(i,k) * cos(del_fd(i) - del_fd(k)) + ...
                B(i,k) * sin(del_fd(i) - del_fd(k)));
            Q_fd(i) = Q_fd(i) + V_fd(i) * V_fd(k) * ...
                (G(i,k) * sin(del_fd(i) - del_fd(k)) - ...
                B(i,k) * cos(del_fd(i) - del_fd(k)));
        end
    end

    % Calculate mismatches

    dP_fd = Psp - P_fd;
    dQ_fd = Qsp - Q_fd;

```

```

mismatch_fd = [dP_fd(2:end); dQ_fd(pq)];
error = Jacob_FD \ mismatch_fd;

del_fd(2:end) = del_fd(2:end) + error(1:nbus-1);
V_fd(pq) = V_fd(pq) + error(nbus:end);

error_avg = abs(mean(error(:)));

if mod(Iter_fd, 10) == 0
    fprintf('  Iteration %d: Max mismatch = %.6e, Avg error = %.6e\n',
...
        Iter_fd, max(abs(mismatch_fd)), error_avg);
end
end
time_fd = toc(tic_fd);
if Iter_fd >= max_iter_fd
    fprintf('\n*** WARNING: Fast Decoupled method did NOT converge!
***\n\n');
end
%% ===== FDLF RESULTS =====
fprintf('\n');
fprintf('=====\n');
if max(abs(mismatch_fd)) < tol_fd
    fprintf('  FAST DECOUPLED: CONVERGED\n');
else
    fprintf('  FAST DECOUPLED: DID NOT CONVERGE\n');
end
fprintf('=====\n');
fprintf('  Iterations: %d\n', Iter_fd);
fprintf('  Max Mismatch: %.6e pu\n', max(abs(mismatch_fd)));
fprintf('  Computation Time: %.6f seconds\n', time_fd);
fprintf('  Time per Iteration: %.6f seconds\n', time_fd/Iter_fd);
fprintf('=====\n\n');
Angles_deg_fd = rad2deg(del_fd);

```

```

fprintf('BUS VOLTAGE AND ANGLE RESULTS (FDLF):\n');

fprintf('-----\n');

fprintf(' Bus | Voltage (pu) | Angle (deg) \n');

fprintf('-----\n');

for i = 1:nbus

    fprintf(' %2d | %7.4f | %9.4f\n', i, V_fd(i),
Angles_deg_fd(i));

end

fprintf('-----\n');

%% ===== FDLF POWER LOSS CALCULATION
=====

fprintf('\nCALCULATING TRANSMISSION LOSSES (FDLF)...\n\n');

Ploss_total_fd = 0;

Qloss_total_fd = 0;

for k = 1:size(linedata, 1)

    i = fb(k);

    j = tb(k);

    R = linedata(k, 3);

    X = linedata(k, 4);

    tap = linedata(k, 6);

    if tap == 0

        tap = 1;

    end

    Z = R + 1i * X;

    y = 1 / Z;

    Vi_fd = V_fd(i) * exp(1i * del_fd(i));

    Vj_fd = V_fd(j) * exp(1i * del_fd(j));

    Iij_fd = (Vi_fd - Vj_fd / tap) * y;

    Iji_fd = (Vj_fd - Vi_fd * tap) * y;

```

```

    Sij_fd = Vi_fd * conj(Iij_fd);
    Sji_fd = Vj_fd * conj(Iji_fd);

    Ploss_total_fd = Ploss_total_fd + real(Sij_fd + Sji_fd);
    Qloss_total_fd = Qloss_total_fd + imag(Sij_fd + Sji_fd);
end

fprintf('=====\n');
fprintf('  SYSTEM LOSSES (FDLF)\n');
fprintf('=====\n');
fprintf('  Active Power Loss:   %.6f pu   (%.3f MW)\n', ...
        Ploss_total_fd, Ploss_total_fd * BMva);
fprintf('  Reactive Power Loss: %.6f pu   (%.3f MVar)\n', ...
        Qloss_total_fd, Qloss_total_fd * BMva);
fprintf('=====\n\n');
%% ===== COMPARISON =====
fprintf('\n=====\n\n');
fprintf('  COMPARISON: NEWTON-RAPHSON vs FAST DECOUPLED\n');
fprintf('=====\n\n');
fprintf('CONVERGENCE & TIMING:\n');
fprintf('-----\n');

fprintf(' Method           | Iterations | Total Time (s) | Time/Iter (s) | Speedup\n');
fprintf('-----\n');

fprintf(' Newton-Raphson    |      %2d   |      %.6f      |      %.6f      | 1.00x\n', ...
        Iter, time_nr, time_nr/Iter);

fprintf(' Fast Decoupled    |      %2d   |      %.6f      |      %.6f      | %.2fx\n', ...
        Iter_fd, time_fd, time_fd/Iter_fd, time_nr/time_fd);
fprintf('-----\n\n');
fprintf('== ALL SIMULATIONS COMPLETE ==\n');

```

RESULTS OF FAST DECOUPLED

B' (B-PRIME) MATRIX - For Real Power/Angle														
=====														
(Neglects resistance and shunt charging)														
Bus 1:	-21.3840	16.9005	0.0000	0.0000	4.4835	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 2:	16.9005	-33.3743	5.0513	5.6715	5.7511	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 3:	0.0000	5.0513	-10.8982	5.8469	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 4:	0.0000	5.6715	5.8469	-42.1801	23.7473	0.0000	4.8895	0.0000	1.8555	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 5:	4.4835	5.7511	0.0000	23.7473	-37.9499	3.9679	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 6:	0.0000	0.0000	0.0000	0.0000	3.9679	-20.5811	0.0000	0.0000	0.0000	0.0000	5.0277	3.9092	7.6764	0.0000
Bus 7:	0.0000	0.0000	0.0000	4.8895	0.0000	0.0000	-19.5490	5.6770	9.0901	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 8:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	5.6770	-5.6770	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 9:	0.0000	0.0000	0.0000	1.8555	0.0000	0.0000	9.0901	0.0000	-26.4209	11.8343	0.0000	0.0000	0.0000	3.6985
Bus 10:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	11.8343	-17.0408	5.2064	0.0000	0.0000	0.0000
Bus 11:	0.0000	0.0000	0.0000	0.0000	0.0000	5.0277	0.0000	0.0000	0.0000	5.2064	-10.2341	0.0000	0.0000	0.0000
Bus 12:	0.0000	0.0000	0.0000	0.0000	0.0000	3.9092	0.0000	0.0000	0.0000	0.0000	0.0000	-8.9122	5.0030	0.0000
Bus 13:	0.0000	0.0000	0.0000	0.0000	0.0000	7.6764	0.0000	0.0000	0.0000	0.0000	0.0000	5.0030	-15.5528	2.8734
Bus 14:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	3.6985	0.0000	0.0000	0.0000	2.8734	-6.5719
=====														
B'' (B-DOUBLE-PRIME) MATRIX - For Reactive Power/Voltage														
=====														
(Neglects resistance, includes shunt charging)														
Bus 1:	-21.3585	16.9005	0.0000	0.0000	4.4835	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 2:	16.9005	-33.3330	5.0513	5.6715	5.7511	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 3:	0.0000	5.0513	-10.8840	5.8469	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 4:	0.0000	5.6715	5.8469	-42.1684	23.7473	0.0000	4.8895	0.0000	1.8555	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 5:	4.4835	5.7511	0.0000	23.7473	-37.9289	3.9679	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 6:	0.0000	0.0000	0.0000	0.0000	3.9679	-20.5811	0.0000	0.0000	0.0000	0.0000	5.0277	3.9092	7.6764	0.0000
Bus 7:	0.0000	0.0000	0.0000	4.8895	0.0000	0.0000	-19.5490	5.6770	9.0901	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 8:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	5.6770	-5.6770	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Bus 9:	0.0000	0.0000	0.0000	1.8555	0.0000	0.0000	9.0901	0.0000	-26.4209	11.8343	0.0000	0.0000	0.0000	3.6985
Bus 10:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	11.8343	-17.0408	5.2064	0.0000	0.0000	0.0000
Bus 11:	0.0000	0.0000	0.0000	0.0000	0.0000	5.0277	0.0000	0.0000	0.0000	5.2064	-10.2341	0.0000	0.0000	0.0000
Bus 12:	0.0000	0.0000	0.0000	0.0000	0.0000	3.9092	0.0000	0.0000	0.0000	0.0000	0.0000	-8.9122	5.0030	0.0000
Bus 13:	0.0000	0.0000	0.0000	0.0000	0.0000	7.6764	0.0000	0.0000	0.0000	0.0000	0.0000	5.0030	-15.5528	2.8734
Bus 14:	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	3.6985	0.0000	0.0000	0.0000	2.8734	-6.5719

fx

Fast Decoupled method initialized...

Tolerance: 0.010000

=====

FAST DECOUPLED: CONVERGED

=====

Iterations: 7

Max Mismatch: 4.769569e-03 pu

Computation Time: 0.034352 seconds

Time per Iteration: 0.004907 seconds

=====

```

-----
Bus | Voltage (pu) | Angle (deg)
-----
 1 |      1.0600   |      0.0000
 2 |      1.0450   |     -4.9655
 3 |      0.9856   |    -12.4247
 4 |      1.0162   |    -10.3078
 5 |      1.0265   |     -8.9051
 6 |      1.0700   |    -14.7598
 7 |      1.0514   |    -13.3750
 8 |      1.0900   |    -13.3750
 9 |      1.0350   |    -14.9944
10 |      1.0337   |    -15.2356
11 |      1.0482   |    -15.1150
12 |      1.0536   |    -15.5954
13 |      1.0476   |    -15.6215
14 |      1.0222   |    -16.2955
-----

```

CALCULATING TRANSMISSION LOSSES (FDLF)...

```

=====
SYSTEM LOSSES (FDLF)
=====

```

```

Active Power Loss:  0.158791 pu  (15.879 MW)
Reactive Power Loss: 0.568189 pu  (56.819 MVar)
=====

```

Discussion of accuracy vs. speed.

=====

CONVERGENCE & TIMING:

Method		Iterations		Total Time (s)		Time/Iter (s)		Speedup
Newton-Raphson		4		0.116294		0.029073		1.00x
Fast Decoupled		7		0.034352		0.004907		3.39x